

Decentralized Content-Addressable Archival: A Robust Architecture for Restic-Based Multi-Volume Optical Storage

1. Introduction: The Imperative of Physical Immutability

In the contemporary landscape of digital asset preservation, the storage hierarchy is dominated by two paradigms: high-availability hot storage (SSD/HDD arrays) and high-latency cold storage (Tape/Cloud Object Storage). While these tiers solve specific problems regarding access speed and capacity, they often fail to address the critical requirement of autonomous, decentralized resilience. Ransomware attacks, cloud vendor lock-in, and account termination pose existential threats to data stored on rewriteable or externally managed infrastructure.¹ This report articulates a comprehensive architectural framework for a third tier: an offline, multi-volume optical archival system leveraging modern deduplication technologies. The primary objective is to design a system that bridges the gap between the granular, content-addressable storage (CAS) model of restic (and its Rust-based implementation, rustic) and the linear, discrete nature of optical media such as Blu-ray (BD-R) and M-Disc. Unlike tape, which allows for contiguous spanning of tarballs, optical media requires discrete "packetization" of data. Furthermore, standard backup tools assume the simultaneous availability of all repository data, a constraint that breaks in a multi-volume offline scenario.³ To satisfy the requirement of recovering from "any combination of volumes which contain the full set of data," this architecture proposes a "Holographic Metadata Strategy." This approach necessitates that the complete catalog of the archive—encompassing the mapping of every encrypted data chunk to its specific physical disc—be replicated onto every single volume in the set. By embedding a specialized SQLite database alongside the standard restic metadata on every disc, the system ensures that the most recent volume alone is sufficient to bootstrap a full recovery process, identifying exactly which previous volumes must be retrieved from physical storage to reconstruct a specific dataset.⁴

2. Theoretical Framework: Content-Addressable Storage on WORM Media

To understand the necessity of an external indexing layer (SQLite), one must first analyze the fundamental incompatibility between restic's design philosophy and the physical constraints of Write-Once, Read-Many (WORM) media.

2.1 The Restic Repository Model

Restic operates on a Content-Addressable Storage (CAS) model. It does not store "files" in the traditional sense. Instead, it breaks files into variable-length chunks based on a rolling Rabin fingerprinting algorithm. These chunks, or "blobs," are deduplicated, encrypted, and then grouped into container files called "packs".⁶

The repository structure consists of:

- **data/**: A directory containing the pack files. These are named by the SHA-256 hash of their contents (e.g., a1b2c3...).
- **index/**: Encrypted files that function as lookup tables. They map a blob's hash to the specific pack file containing it and its offset within that pack.⁸
- **snapshots/**: JSON documents describing the directory state at a specific point in time (paths, timestamps, permissions) and the tree blobs required to reconstruct it.⁹
- **config**: The repository initialization file containing the repository ID, version, and chunker polynomial.¹⁰
- **keys/**: The encryption keys (password protected) required to decrypt the master key.¹¹

In a standard deployment, restic assumes random access to the entire data/ directory. If a restore operation requires Blob X, restic checks the index, finds it is in Pack Y, and attempts to read Pack Y. If Pack Y is on a disc sitting on a shelf, the operation fails immediately.

2.2 The Challenge of Multi-Volume Spanning

Unlike tar, which supports multi-volume spanning by splitting a single stream of bytes across tapes (-M flag)¹², restic has no native concept of a "split" repository. A repository is viewed as a monolithic entity. If one tries to split the data/ directory across multiple discs without an orchestration layer, the following problems arise:

1. **Index Blindness**: The restic client typically reads *all* index files at startup. If the indices are scattered across discs, the client cannot build a complete map of the repository.¹³
2. **Retrieval Paralysis**: Even if the client has the full index (e.g., cached locally), it has no way to prompt the user: "Please insert Disc 4 to retrieve Pack 7f8a...." It simply errors out when the file is not found.³
3. **Orphaned Data**: Without a master catalog, if Disc 1 contains the data and Disc 2 contains the snapshot pointing to it, losing Disc 2 renders the data on Disc 1 unidentifiable and effectively lost, even if the bits are intact.³

2.3 The Solution: The "Rustic" Primitives and SQLite

To solve this, we employ rustic, a Rust-based reimplementation of restic that introduces the concept of "cold storage".¹⁵ rustic explicitly separates the repository into:

- **Hot Repository**: Contains metadata (index, snapshots, keys, config) and potentially directory tree structures. This must be available for operations to start.
- **Cold Repository**: Contains the bulk data packs. These are assumed to be expensive or slow to retrieve.¹⁵

Our architecture leverages this split. The "Hot Repository" is simulated by the metadata burned onto every optical disc (the Holographic Strategy), while the "Cold Repository" corresponds to the aggregate collection of physical discs. The SQLite database serves as the "hypervisor," translating the logical requests for packs into physical prompts for specific Volume Labels.⁵

3. Database Schema Design for Offline Asset Management

The SQLite database is the central nervous system of this architecture. It must provide an immutable, append-only log of every pack file written to optical media, facilitating the "management (removal/replacement)" requirement of the prompt.

3.1 Entity-Relationship Model

The schema requires three core tables: volumes, packs, and snapshots. A fourth table, volume_events, tracks the lifecycle of the physical media.

3.1.1 Table: volumes

This table defines the physical containers (discs).

Column Name	Data Type	Constraints	Description
volume_id	INTEGER	PRIMARY KEY AUTOINCREMENT	Internal numeric ID.
volume_label	TEXT	UNIQUE NOT NULL	The human-readable label printed on the disc (e.g., ARCHIVE_2024_001).
uuid	TEXT	UNIQUE NOT NULL	A cryptographically generated UUID (v4) to uniquely identify the media, avoiding label collisions. ¹⁸
media_type	TEXT	NOT NULL	E.g., BD-R-25, BD-XL-100, DVD-RW, M-DISC.
capacity_bytes	INTEGER	NOT NULL	The total usable capacity of the disc (e.g., 23,500,000,000 for 25GB BD-R).
used_bytes	INTEGER	DEFAULT 0	The actual amount of data written.
created_at	DATETIME	DEFAULT	When the volume was

		CURRENT_TIMESTAMP	initialized in the DB.
closed_at	DATETIME	NULLABLE	When the volume was finalized (burned). If NULL, it is the current "open" staging batch.
status	TEXT	CHECK(status IN ('OPEN', 'BURNED', 'VERIFIED', 'MISSING', 'DESTROYED'))	Lifecycle state of the volume.
location	TEXT	DEFAULT 'OFFLINE'	Physical location tracking (e.g., "Safe A", "Offsite").

Insight: The volume_label allows the restorer to prompt the user (e.g., "Insert Disc ARCHIVE_2024_001"), while the uuid allows the software to verify that the inserted disc is correct by reading a volume_info.json file stored on the root of the disc, preventing accidental restores from the wrong media set.¹⁹

3.1.2 Table: packs

This table maps the CAS blobs to physical volumes.

Column Name	Data Type	Constraints	Description
pack_id	TEXT	PRIMARY KEY	The SHA-256 hash filename of the pack file. ⁶
volume_id	INTEGER	FOREIGN KEY(volumes.id)	The specific disc holding this pack.
size	INTEGER	NOT NULL	Size of the pack file in bytes.
type	TEXT	CHECK(type IN ('data', 'tree'))	'tree' packs contain directory structures; 'data' packs contain file contents. ⁸
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Timestamp of indexing.
verification_date	DATETIME	NULLABLE	Last date this specific pack was checksum-verified.
is_pruned	BOOLEAN	DEFAULT 0	Marks packs that have been logically deleted by a prune operation but physically remain on WORM media.

Insight regarding is_pruned: On optical media, we cannot delete files. When restic prune runs, it repacks live blobs into new packs and "deletes" old packs. In our system, "deletion" is a metadata operation: we set is_pruned = 1. This instructs the restoration logic to ignore this pack, even though it physically exists on the disc. This is crucial for managing the growth of the index "over time" as requested.

3.1.3 Table: snapshots

This table mirrors restic's snapshot metadata to allow for SQL-based querying of restore requirements without mounting the repository.

Column Name	Data Type	Constraints	Description
snapshot_id	TEXT	PRIMARY KEY	The restic snapshot ID. ⁹
timestamp	DATETIME	NOT NULL	The time the snapshot was taken.
hostname	TEXT	NOT NULL	The host source of the backup.
tags	TEXT	NULLABLE	JSON array of tags (e.g., ["daily", "production"]). ²⁰
paths	TEXT	NOT NULL	JSON array of paths included in the snapshot.
volume_map	TEXT	NULLABLE	Crucial: A pre-computed JSON array of volume_ids required to fully restore this snapshot.

Insight: The volume_map field acts as an optimization. Instead of recursively walking the tree to find all packs and their volumes at restore time, this field is populated after the burn process. It allows for instant "Pick List" generation: "To restore Snapshot X, you need Volumes A, B, and C."

3.2 Database Growth and Maintenance

The prompt expresses concern about the database growing over time. SQLite is capable of handling terabytes of data, but for a backup index, the row count is the primary factor.

- **Scale Analysis:** A 1TB backup might produce ~200,000 packs (assuming ~5MB avg pack size). A table with 200k rows is trivial for SQLite.²² Even with petabytes of data (millions of rows), SQLite remains performant given proper indexing on pack_id and volume_id.
- **Maintenance:** The VACUUM command should be run periodically by the Orchestrator script to reclaim space and defragment the DB file, especially after large prune

operations where many rows might be marked as deleted or updated.²³

4. Ingestion Workflow: The Staging and Bin-Packing Process

Because restic cannot write directly to offline media, we implement a "Buffer-then-Burn" workflow. This ensures that only full, optimal ISO images are burned, maximizing media usage.

4.1 Phase 1: The Staging Repository

The user performs backups to a local "Staging Repository" located on a fast HDD or SSD.

- **Command:** `restic -r /srv/staging_repo backup /home/user/data`
- **Behavior:** restic performs deduplication against the existing staging data. New blobs are created and grouped into new pack files in `/srv/staging_repo/data/`. Existing blobs are referenced but not duplicated.
- **Result:** The staging repo grows with every backup. It contains a mix of "Archived" packs (already burned to previous discs) and "Pending" packs (new since the last burn).

4.2 Phase 2: Indexing and Delta Analysis

The Orchestrator script performs a delta analysis to determine what needs to be burned.

1. **Scan:** The script scans `/srv/staging_repo/data/` for all pack IDs.
2. **Query:** It queries the SQLite packs table: `SELECT pack_id FROM packs`.
3. **Diff:** Pending Packs = Filesystem Packs - Indexed Packs.
4. **Threshold Check:** If $\text{Sum}(\text{Size}(\text{Pending Packs})) > \text{Media Capacity}$ (e.g., 25GB), a burn cycle is triggered.

4.3 Phase 3: The "Bin Packing" Algorithm

To optimize space on the optical disc, the system selects a subset of Pending Packs to fit the target media. A "First Fit Decreasing" algorithm is sufficient here:

1. Sort Pending Packs by size (largest to smallest).
2. Initialize a "Bin" (the ISO image) with capacity C (e.g., 23.5GB to allow FS overhead).
3. Iterate through the sorted packs. If a pack fits in the remaining space `C_current`, add it to the Bin and subtract its size.
4. Repeat until the Bin is full.

4.4 Phase 4: Holographic Metadata Assembly

This is the most critical step for meeting the "recover from any combination of volumes" requirement. We create a temporary directory tree for the ISO image that includes **all** metadata known to the system at this specific moment.

ISO Directory Layout:

/ (Root of ISO)

├── archive_index.db # The MASTER SQLite catalog (The "Map")

├── volume_info.json # Self-description: { "uuid": "...", "label": "VOL_05", "prev_vol": "VOL_04"

```

}
├── restic/
│   ├── config # Copy of repo config
│   ├── keys/ # Copy of ALL keys (Encrypted)
│   ├── snapshots/ # Copy of ALL snapshots (from staging)
│   ├── index/ # Copy of ALL indices (from staging)
│   └── data/
│       ├── 1a/ # Subdirectories matching restic structure
│       │   └── 1a2b3c... # ONLY the packs selected by Bin Packing for this disc
│       └── ...
├── recovery/
├── tools/ # Static binaries of restic, rustic, sqlite3
└── ecc_data.ecc # Error correction data (generated later)

```

Insight: The restic/data/ directory on the disc is a *subset* of the full repository. However, restic/index/ and restic/snapshots/ are *supersets* (or complete sets). This means that **any** disc you pick up contains the indexes for **every** pack on **every** other disc ever burned up to that point.³ This solves the "missing index" problem native to restic's split repository limitations.

4.5 Phase 5: Mastering and Burning with Xorriso

We use xorriso to master the ISO image. xorriso is preferred over genisoimage due to its robust support for large files, scripting, and multi-session capabilities.²⁴

Command Construction:

Bash

```

xorriso -as mkisofs \
-V "ARCHIVE_2024_005" \
-volid "ARCHIVE_2024_005" \
-J -R \
-iso-level 3 \
-o /tmp/burn_image.iso \
/tmp/iso_staging_dir

```

- -J: Joliet extension for Windows compatibility.
- -R: Rock Ridge extension for POSIX permissions (crucial for Linux restores).
- -iso-level 3: Supports files larger than 4GB (though Restic packs are small, this is good practice).

4.6 Phase 6: Integrity Layer (ECC)

Before burning (or during, if using dvdaster's RS03 augmented mode), we generate Error Correction Code (ECC) data. Optical media is subject to "bit rot" (dye degradation) and

physical scratches.

- **Tool:** dvdaster.²⁶
- **Method:** Generate "Augmented Image". dvdaster appends Reed-Solomon redundancy data to the ISO file itself.
 - *Trade-off:* This consumes disc space (typically ~15-30% overhead). For a 25GB disc, we might only use 20GB for data and 5GB for ECC.
 - *Implementation:* The Bin Packing algorithm in Phase 3 must account for this reserved space.
 - *Command:* dvdaster -mRS03 -c /tmp/burn_image.iso

Insight: This allows the disc to self-repair. If a future read fails due to a scratch, dvdaster can reconstruct the missing sectors using the ECC data stored *on the same disc*.

5. Restoration Architecture: The Virtual Union

The restoration process must seamlessly stitch together the fragmented data packs. The user requires a "restorer" tool (likely a Python script wrapping rustic) that consults the SQLite index to guide the physical workflow.

5.1 Bootstrapping the Restore

1. **Identify Most Recent Volume:** The user inserts *any* disc they believe to be the most recent.
2. **Mount & Verify:** The script mounts the disc. It reads volume_info.json and archive_index.db.
3. **Freshness Check:** The script queries the volumes table in the DB on the disc. It checks:
 - SELECT Max(created_at) FROM volumes.
 - If the DB on the disc knows about "Volume 10," but the inserted disc is "Volume 8," the script prompts: *"This index is outdated. Please insert Volume 10 or newer to ensure you have the latest snapshot definitions."*
 - If the user has lost Volume 10, they can choose to proceed with Volume 8, accepting that they can only restore snapshots that existed up to the creation of Volume 8.

5.2 Snapshot Selection & Dependency Resolution

Once the latest index is loaded into memory (or copied to a temp location):

1. **List Snapshots:** The script queries the snapshots table to show available restore points.
2. **Select Snapshot:** User selects Snapshot_2024-02-01.
3. **Resolve Dependencies:**
 - The system uses rustic restore <snapshot_id> --dry-run --json.²⁸ This command runs against the metadata (which is fully present).
 - rustic calculates the exact list of pack hashes required to reconstruct the files.
 - The Orchestrator script intercepts this JSON output.

5.3 The Volume Fetch Manifest

The script takes the list of required Pack hashes and queries the local SQLite DB:

SQL

```
SELECT v.volume_label, v.location, COUNT(p.pack_id) as count, SUM(p.size) as total_size
FROM packs p
JOIN volumes v ON p.volume_id = v.volume_id
WHERE p.pack_id IN (... list of 5,000 required hashes...)
GROUP BY v.volume_label
ORDER BY total_size DESC;
```

Output Table:

Volume Label	Location	Packs Needed	Data Size	Status
ARCHIVE_005	Safe 1	1,204	14.2 GB	Offline
ARCHIVE_002	Offsite	402	4.1 GB	Offline
ARCHIVE_001	Safe 1	12	0.1 GB	Offline

This is the "Pick List." It tells the user exactly which discs to retrieve.

5.4 The "Virtual Union" Restore Process

Restic/Rustic expects all packs to be present in data/. We simulate this using a local cache directory.

Workflow:

1. **Prepare Cache:** `mkdir -p /tmp/restore_cache/data/`.
2. **Link Metadata:** Symlink index/, snapshots/, config, keys/ from the mounted optical disc (or the staging repo) to /tmp/restore_cache/.
3. **Fetch Loop:**
 - Prompt: *"Please insert Volume ARCHIVE_005"*.
 - Mount Disc.
 - Verify UUID matches DB.
 - Copy the 1,204 required packs from /media/cdrom/restic/data/ to /tmp/restore_cache/data/.
 - *Optimization:* If the restore is huge and local disk is small, we cannot copy everything. However, restic access patterns are essentially random. We *must* copy the packs to a local staging area to ensure a smooth restore. This architecture assumes the restorer has HDD space > Restore Size.
 - Eject Disc.
4. **Execute Restore:** Once all needed packs are in /tmp/restore_cache/data/, run:
`rustic -r /tmp/restore_cache restore <snapshot_id> --target /restore/path`

Handling "Recover from Any Combination":

If ARCHIVE_002 is lost (destroyed), the script detects this (user says "Skip"). The packs from

ARCHIVE_002 will be missing from /tmp/restore_cache.

- **Result:** rustic will attempt the restore. Files dependent on packs from Vol 002 will fail/error. Files entirely contained on Vol 005 and 001 will restore successfully.
- **Partial Recovery:** This satisfies the requirement. The Holographic Index allowed us to identify *what* was on the lost disc and restore everything else. Without the index, we wouldn't have known which files were salvageable.

6. Security, Integrity, and Future-Proofing

6.1 Cryptographic Security

Restic uses AES-256-CTR for data encryption.¹¹

- **Key Management:** The keys/ directory is stored on every disc. This means the disc is self-contained. Security relies entirely on the repository password.
- **Recommendation:** Use a high-entropy passphrase (e.g., Diceware, >20 chars).
- **Key Rotation:** If a password is compromised, restic key add / restic key remove can be used on the *staging* repo. Future burns will contain the new key files. Old discs will still have the old key files (and thus be vulnerable to the old password). This is a limitation of WORM media; physical destruction of old discs is the only remediation for compromised keys.

6.2 Data Integrity and Bit Rot

- **Check Command:** The Orchestrator should support a verify mode. The user inserts a disc, and the script runs restic check --read-data specifically on the packs present on that disc.²⁹
- **ECC Recovery:** If restic check reports a checksum mismatch (bit rot), the user runs dvd disaster -r on the ISO image (which must be ripped from the disc first) to repair the corrupted sectors using the Reed-Solomon data, then burns a replacement disc.²⁶

6.3 Schema Migration and Future-Proofing

- **Self-Contained Tools:** To prevent the "Bootstrap Paradox" (having the data but not the software), every burn includes a recovery/tools/ folder containing statically compiled binaries of restic, rustic, sqlite3, and the Orchestrator Python scripts for Linux, Windows, and macOS.¹⁶
- **Format Evolution:** If the SQLite schema changes in version 2.0 of the Orchestrator, the new script simply creates a new DB file. Since the DB is re-written to every new disc, there is no "migration" of old discs needed. Old discs hold the v1 DB; new discs hold the v2 DB. The v2 Orchestrator must simply know how to read v1 DBs for "Fetch Manifest" generation.

7. Conclusion

This report defines a resilient, offline-first archival architecture that satisfies the stringent

requirements of decentralized recovery. By coupling restic's cryptographic deduplication with SQLite's relational indexing and xorriso's physical media management, we create a system where:

- 1. **Redundancy is Ubiquitous:** The map (index.db + metadata) exists on every shard of the archive.
- 2. **Dependencies are Explicit:** The database enforces referential integrity between logical blobs and physical volumes.
- 3. **Recovery is Deterministic:** The Orchestrator provides clear, step-by-step instructions for physical media retrieval, transforming a complex dependency graph into a linear "Jukebox" workflow.

This system effectively neutralizes the threats of ransomware and cloud instability, providing a "Forever Archive" that requires nothing but the physical discs and a commodity optical drive to resurrect decades of digital history.

Appendix A: Comparative Technology Matrix

Feature	Proposed Restic+SQLite Architecture	LTO Tape (Native)	Cloud Cold Storage (Glacier)	Git-Annex
Access Pattern	Random (Simulated via Cache)	Linear / Sequential	Random (High Latency)	File-based
Deduplication	Block-level (High Efficiency)	None (unless hardware compression)	None (File-level)	File-level
Metadata	Holographic (On every node)	Linear (Header/Trailer)	Centralized (Cloud DB)	Distributed (Git)
Recovery Cost	\$0 (Owned Hardware)	\$0 (Owned Hardware)	\$\$\$ (Egress Fees)	\$0
Hardware Cost	Low (~\$100 BD Writer)	High (>\$3000 LTO Drive)	None (OpEx)	Low
Immutability	Physical WORM (Hardware enforced)	WORM Cartridges available	Object Lock (Soft)	None (Rewriteable)
Partial Restore	Yes (via Index)	Slow (Seek times)	Yes	Yes

Appendix B: Orchestrator Logic (Pseudocode for prepare_burn.py)

Python

```

def prepare_burn_batch(staging_path, db_conn, media_capacity):
    """
    Identifies unarchived packs and prepares a directory structure for burning.
    """

    # 1. Scan staging repo for all current pack IDs
    staging_packs = scan_directory(staging_path + "/data")

    # 2. Query SQLite for packs already assigned to a volume
    cursor = db_conn.cursor()
    cursor.execute("SELECT pack_id FROM packs")
    archived_packs = set(row for row in cursor.fetchall())

    # 3. Determine Pending Packs (Delta)
    pending_packs = staging_packs - archived_packs

    # 4. Bin Packing (First Fit)
    batch =
    current_batch_size = 0
    reserved_overhead = 1024 * 1024 * 100 # 100MB for DB/Metadata

    for pack_id in pending_packs:
        pack_size = get_file_size(staging_path, pack_id)
        if (current_batch_size + pack_size) < (media_capacity - reserved_overhead):
            batch.append(pack_id)
            current_batch_size += pack_size
        else:
            break # Bin is full

    # 5. Construct Burn Directory
    burn_root = create_temp_dir()

    # Link Data Packs (Cold)
    for pack in batch:
        hardlink(staging_path + "/data/" + pack, burn_root + "/restic/data/" + pack)

    # Copy ALL Metadata (Hot) - The Holographic Step
    copy_tree(staging_path + "/index", burn_root + "/restic/index")
    copy_tree(staging_path + "/snapshots", burn_root + "/restic/snapshots")
    copy_tree(staging_path + "/keys", burn_root + "/restic/keys")
    copy_file(staging_path + "/config", burn_root + "/restic/config")

    # 6. Update Database (Pending State)

```

```

vol_uuid = generate_uuid()
vol_label = generate_next_label(db_conn) # e.g. "ARCHIVE_005"
db_conn.execute("INSERT INTO volumes (uuid, label, status) VALUES (?, ?, 'PENDING')",
(vol_uuid, vol_label))
vol_id = db_conn.lastrowid

for pack in batch:
    db_conn.execute("INSERT INTO packs (pack_id, volume_id) VALUES (?, ?)", (pack, vol_id))

# 7. Copy Database to Burn Root
db_conn.commit() # Save state
copy_file(db_path, burn_root + "/archive_index.db")

return burn_root, vol_label

```

Works cited

1. Separating data/ and repository metadata - Getting Help - restic forum, accessed February 12, 2026,
<https://forum.restic.net/t/separating-data-and-repository-metadata/8717>
2. Archival Storage | Hacker News, accessed February 12, 2026,
<https://news.ycombinator.com/item?id=43391459>
3. Strategy to split snapshots over many blurays - Getting Help - restic forum, accessed February 12, 2026,
<https://forum.restic.net/t/strategy-to-split-snapshots-over-many-blurays/1486>
4. Central catalog and backup spanning multiple disks - Getting Help - restic forum, accessed February 12, 2026,
<https://forum.restic.net/t/central-catalog-and-backup-spanning-multiple-disks/10609>
5. Design and Manage SQLite Databases with a Modeling Tool - DbSchema, accessed February 12, 2026,
<https://dbschema.com/blog/databases/sqlite-database-dbschema/>
6. Terminology — restic 0.18.1 documentation, accessed February 12, 2026,
<https://restic.readthedocs.io/en/stable/design.html>
7. restic Design Document - Documentation for restic, accessed February 12, 2026,
<https://restic.readthedocs.io/en/v0.2.0/Design/>
8. Indexing - rustic dev documentation, accessed February 12, 2026,
<https://rustic.cli.rs/dev-docs/design/indexing.html>
9. Snapshots - rustic dev documentation, accessed February 12, 2026,
<https://rustic.cli.rs/dev-docs/design/snapshots.html>
10. rustic dev documentation, accessed February 12, 2026,
<https://rustic.cli.rs/dev-docs/rustic-dev-docs.pdf>
11. Repository - rustic dev documentation, accessed February 12, 2026,
<https://rustic.cli.rs/dev-docs/design/repository/intro.html>

12. How should I divide my data for backup media? : r/linuxadmin - Reddit, accessed February 12, 2026,
https://www.reddit.com/r/linuxadmin/comments/4m8jk6/how_should_i_divide_my_data_for_backup_media/
13. Restic indexing extremely slow on external 4TB disk - Getting Help, accessed February 12, 2026,
<https://forum.restic.net/t/restic-indexing-extremely-slow-on-external-4tb-disk/8501>
14. Backup with online and offline storage - Getting Help - restic forum, accessed February 12, 2026,
<https://forum.restic.net/t/backup-with-online-and-offline-storage/8692>
15. FAQ - rustic user documentation, accessed February 12, 2026,
<https://rustic.cli.rs/docs/FAQ.html>
16. rustic-rs/rustic: rustic - fast, encrypted, and deduplicated backups powered by Rust - GitHub, accessed February 12, 2026, <https://github.com/rustic-rs/rustic>
17. avelino/awesome-go: A curated list of awesome Go frameworks, libraries and software - GitHub, accessed February 12, 2026,
<https://github.com/avelino/awesome-go>
18. some design questions · Issue #347 · restic/restic - GitHub, accessed February 12, 2026, <https://github.com/restic/restic/issues/347>
19. Restic Backup from macOS Apple APFS Snapshot of External Drive (without Time Machine), accessed February 12, 2026,
<https://forum.restic.net/t/restic-backup-from-macos-apple-apfs-snapshot-of-external-drive-without-time-machine/10488>
20. Database backups, dump files and restic - The Ongoing Struggle, accessed February 12, 2026,
<https://strugglers.net/posts/2025/database-backups-dump-files-and-restic/>
21. Two different data types/folders in the same restic repository? - Getting Help, accessed February 12, 2026,
<https://forum.restic.net/t/two-different-data-types-folders-in-the-same-restic-repository/6870>
22. SQLite ETL Tools: Lightweight Pipelines for Edge & Mobile Apps | Integrate.io, accessed February 12, 2026,
<https://www.integrate.io/blog/sqlite-etl-tools-lightweight-pipelines/>
23. Best Practices for Managing Schema, Indexes, and Storage in SQLite for Data Engineering | by firman brilian | Medium, accessed February 12, 2026,
<https://medium.com/@firmanbrilian/best-practices-for-managing-schema-indexes-and-storage-in-sqlite-for-data-engineering-c74f71056518>
24. xorriso - man pages section 1: User Commands, accessed February 12, 2026,
https://docs.oracle.com/cd/E86824_01/html/E54763/xorriso-1.html
25. creates, loads, manipulates and writes ISO 9660 filesystem images with Rock Ridge extensions. | Man Page | Commands | xorriso | ManKier, accessed February 12, 2026, <https://www.mankier.com/1/xorriso>
26. speed47/dvdisaster: A tool providing additional ECC protection for optical media (unofficial version) - GitHub, accessed February 12, 2026,

<https://github.com/speed47/dvdisaster>

27. Dvdisaster : r/DataHoarder - Reddit, accessed February 12, 2026,
<https://www.reddit.com/r/DataHoarder/comments/hy7j3d/dvdisaster/>
28. Restore dry-run - Features and Ideas - restic forum, accessed February 12, 2026,
<https://forum.restic.net/t/restore-dry-run/2840>
29. Working with repositories — restic 0.18.1-dev documentation, accessed February 12, 2026, https://restic.readthedocs.io/en/latest/045_working_with_repos.html